

```

"""
A* Search Implementation in Python

Statement: Implement the A* search algorithm for traversing or
searching tree or graph data structures.
"""

import heapq

def a_star_search(graph: dict, start: str, goal: str, heuristic: dict) -> list:
    """
    A* Search algorithm implementation

    Args:
        graph: Dictionary representing the graph (adjacency list)
        start: Starting node for A* search
        goal: Goal node to reach
        heuristic: Dictionary containing heuristic values for each node

    Returns:
        List of nodes representing the path from start to goal
    """
    open_list = [] # Priority queue to store nodes to explore
    heapq.heappush(open_list, (0, start)) # Push the starting node with initial cost 0

    g_costs = {start: 0} # Dictionary to store g(n) costs
    parent = {start: None} # Dictionary to store parent nodes for path reconstruction

    while open_list:
        current_f_cost, current_node = heapq.heappop(open_list) # Get the node with the lowest f(n) cost

        if current_node == goal: # If we have reached the goal, reconstruct the path
            path = [] # List to store the path from start to goal
            while current_node: # Backtrack from the goal to the start using the parent dictionary
                path.append(current_node) # Add the current node to the path
                current_node = parent[current_node] # Move to the parent node
            return path[::-1] # Return reversed path

        for neighbor, cost in graph[current_node]: # Explore neighbors of the current node
            new_g_cost = g_costs[current_node] + cost # Calculate tentative g(n) cost

            if neighbor not in g_costs or new_g_cost < g_costs[neighbor]: # If this path to neighbor is better
                g_costs[neighbor] = new_g_cost # Update g(n) cost for the neighbor
                f_cost = new_g_cost + heuristic.get(neighbor, float('inf')) # Calculate f(n) cost
                heapq.heappush(open_list, (f_cost, neighbor)) # Push the neighbor into the open list
                parent[neighbor] = current_node # Update parent for path reconstruction

    return [] # Return empty path if no path is found

def main():
    """
    Main function to demonstrate A* search on a sample graph.

    Args:
        None

    Returns:
        None
    """
    # Sample graph as adjacency list
    graph = {
        'A': [('B', 1), ('C', 3)],
        'B': [('D', 3), ('E', 1)],
        'C': [('F', 5)],
        'D': [],
    }

```

```
    'E': [('F', 2)],
    'F': []
}

# Sample heuristic values
heuristic = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 0
}

start = 'A'
goal = 'F'

path = a_star_search(graph, start, goal, heuristic)
print(f"Path from {start} to {goal}: {path}")

if __name__ == "__main__":
    main()
```

```
# Output:
# Path from A to F: ['A', 'B', 'E', 'F']
```